

Digital Image Processing

Exercises & Worked Solutions (លំហាត់ និងដំណោះស្រាយ)

Topics: spatial filtering, color models (HSI/HSV/CMYK), edge detection, and mathematical morphology

Exercise 1 : Spatial filter & Boundary Handling

□ Problem Statement

You are given a 3x3 grayscale image segment I and a 3x3 box filter kernel K defined below.

$$I = \begin{bmatrix} 10 & 20 & 30 \\ 40 & 50 & 60 \\ 70 & 80 & 90 \end{bmatrix} \quad \text{and} \quad H = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

1. Calculate the filtered pixel value at the center of the image (Value 50) using valid padding (boundary extension needed).
2. Calculate the filtered pixel value for the top-left pixel (value 10) by applying zero-padding (assume all pixel values outside the image boundary are 0)

✓ Solution

នេះជាដំណោះស្រាយដល់ដំណើរការសម្រាប់លំហាត់ទី១ (Spatial Filter & Boundary Handling) របស់អ្នក៖

1. គណនាតម្លៃ Pixel ថ្មីនៅចំកណ្តាល (តម្លៃ 50) ដោយប្រើ Valid Padding

ពាក្យថា **Valid Padding** នៅក្នុង Image Processing (ឬពេលខ្លះហៅថា No Padding)

មានន័យថាយើងមិនអនុញ្ញាតឱ្យ Kernel ហៀរចេញក្រៅព្រំដែននៃរូបភាពឡើយ

ពេលគឺយើងគណនាបានតែត្រង់ចំណុចណាដែល Kernel អាចគ្របដណ្តប់ពីលើរូបភាពបានពេញលេញ 3×3 តែម្តង។

ដូចនេះ នៅពេលយើងដាក់ Kernel H ឱ្យចំកណ្តាលរូបភាព (ចំលេខ 50) វានឹងត្រួតស៊ីគ្នាជាមួយ Matrix ទាំងស្រុង ដោយមិនចាំបាច់ថែមតែម្សៅទាំងអស់។

រូបមន្ត និងជំហានគណនា៖ យើងយកធាតុនីមួយៗនៃរូបភាព ទៅគុណនឹងធាតុចំទីតាំងជាមួយគ្នានៃ Kernel រួចបូកសរុបបញ្ចូលគ្នា៖

$$Result = \frac{1}{9} \times [(10 \times 1) + (20 \times 1) + (30 \times 1) + (40 \times 1) + (50 \times 1) + (60 \times 1) + (70 \times 1) + (80 \times 1) + (90 \times 1)]$$

$$Result = \frac{1}{9} \times [10 + 20 + 30 + 40 + 50 + 60 + 70 + 80 + 90]$$

$$Result = \frac{1}{9} \times [450] = 50$$

ចម្លើយសំណួរទី១៖ តម្លៃផ្ទាំងរូបភាពចម្លង (Filtered value) នៅចំកណ្តាលគឺ **50**។

2. គណនាតម្លៃ Pixel ថ្មីនៅជ្រុងលើខាងឆ្វេង (តម្លៃ 10) ដោយប្រើ Zero-Padding

នៅពេលយើងចង់គណនាត្រង់ចំណុចតែម ឬជ្រុង ដូចជាលេខ 10 នេះ Kernel ទំហំ 3×3 នឹងត្រូវហៀរចេញទៅក្រៅព្រំដែនរូបភាព។ តាមលក្ខខណ្ឌ Zero-Padding យើងត្រូវសន្មតថា រាល់តំបន់ដែលហៀរចេញទៅក្រៅរូបភាពដើម គឺត្រូវជំនួសដោយលេខ 0 ទាំងអស់។

នៅពេលយើងយក Kernel ទៅដាក់ឱ្យចំកណ្តាលលេខ 10 នោះតំបន់ដែលគ្របដណ្តប់នឹងមានទម្រង់ Matrix ដូចខាងក្រោម៖

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 10 & 20 \\ 0 & 40 & 50 \end{bmatrix}$$

- ជំហានគណនា៖ យើងយកធាតុដែលត្រូវគ្នាគុណបញ្ចូលគ្នា (យើងអាចដកលេខ 0 ចេញដើម្បីឱ្យគណនាលឿន)៖

$$\text{Result} = \frac{1}{9} \times [(0 \times 1) + (0 \times 1) + (0 \times 1) + (0 \times 1) + (10 \times 1) + (20 \times 1) + (0 \times 1) + (40 \times 1) + (50 \times 1)]$$

$$\text{Result} = \frac{1}{9} \times [10 + 20 + 40 + 50]$$

$$\text{Result} = \frac{1}{9} \times [120] \approx 13.33$$

ចម្លើយសំនួរទី២៖ តម្លៃផ្ទាំងរូបភាពចម្លងនៅជ្រុងលើខាងឆ្វេងគឺ 13.33 (ឬ 13 ប្រសិនបើបង្កកជាចំនួនគត់ 8-bit pixel)។

Exercise 2 : RGB to HSV intensity extraction

Problem Statement

A digital color image contains a pixel with the RGB Value: $R = 100$, $G = 150$, and $B = 200$ (on a standard 8-bit scale from 0 to 255).

1. Compute the intensity component (I) of the pixel in the HSI (Hue, Saturation, Intensity) color model.
2. Normalize the RGB values to the range $[0,1]$ and compute the saturation (S) component using the standard formula:

$$S = 1 - \frac{3}{R + G + B} [\min(R, G, B)]$$

Solution

នេះជាការកែសម្រួល និងពន្យល់លម្អិតឡើងវិញសម្រាប់ Exercise 2 (RGB to HSI Conversion)៖

សំណួរទី ១៖ គណនាតម្លៃ Intensity component (I)

នៅក្នុងប្រព័ន្ធពណ័ HSI សមាសភាគ Intensity (I) គឺជាមធ្យមភាគនព្វន្ឋ (Arithmetic Average) នៃតម្លៃពណ័ទាំងបី។ វាគណនាឱ្យកម្រិតពន្លឺសរុបដោយមិនគិតពីជាតិពណ័ឡើយ។

- រូបមន្ត៖ $I = \frac{R+G+B}{3}$
- ការគណនា៖ ទិន្នន័យដែលបានផ្តល់ឱ្យ៖ $R = 100, G = 150, B = 200$

$$I = \frac{100 + 150 + 200}{3} = \frac{450}{3} = 150$$

ចម្លើយទី ១៖ តម្លៃ Intensity component (I) គឺ **150** (នៅលើកម្រិត 8-bit scale ដដែល)។

សំណួរទី ២៖ ធ្វើ Normalization ទៅលើចន្លោះ $[0, 1]$ រួចគណនា Saturation component (S)

ដើម្បីប្រើប្រាស់រូបមន្តនេះបានត្រឹមត្រូវ ជំហានដំបូងបំផុតគឺត្រូវធ្វើ Normalization ទៅលើតម្លៃ R, G, B ពីកម្រិត 8-bit (0 ដល់ 255) ឱ្យទៅនៅក្នុងចន្លោះ $[0,1]$ ជាមុនសិន ដោយយកតម្លៃនីមួយៗទៅចែកនឹង 255។

- ជំហានទី ១៖ គណនាតម្លៃ Normalized R, G, B

$$R_{\text{norm}} = \frac{100}{255} \approx 0.3922, \quad G_{\text{norm}} = \frac{150}{255} \approx 0.5882, \quad B_{\text{norm}} = \frac{200}{255} \approx 0.7843$$
- ជំហានទី ២៖ ស្វែងរកតម្លៃតូចបំផុត $\min(R, G, B)$

$$\min(0.3922, 0.5882, 0.7843) = 0.3922 \quad (\text{គឺជាតម្លៃរបស់ } R_{\text{norm}})$$

- ជំហានទី ៣៖ គណនាតម្លៃ Saturation (S) តាមរូបមន្ត

$$S = 1 - \frac{3}{0.3922 + 0.5882 + 0.7843} \cdot (0.3922)$$

គណនាម៉ាត្រាតម្លៃបែងចែក ($R + G + B$): $0.3922 + 0.5882 + 0.7843 = 1.7647$

$$S = 1 - \frac{3}{1.7647} \cdot (0.3922) = 1 - (1.7000 \cdot 0.3922) = 1 - 0.6667 = 0.3333$$

(សម្គាល់៖ ប្រសិនបើអ្នកយកតម្លៃ 8-bit ទៅជំនួសក្នុងរូបមន្តផ្ទាល់ដោយមិនធ្វើ Normalization ក៏ទទួលបានលទ្ធផល $S = 0.3333$ ដូចគ្នាដែរ ពីព្រោះផលធៀបលេខចែកនឹងសម្រួលគ្នាអស់ ប៉ុន្តែតាមគោលការណ៍គ្រឹះនៃ HSI Model គឺត្រូវតែធ្វើ Normalization ទៅជា $[0,1]$ ជាមុនសិន)
ចម្លើយទី ២៖ តម្លៃ Saturation component (S) គឺ **0.3333** (ឬស្មើនឹង **33.33%**)។

Exercise 3 : CMYK color complement Calculate

Problem Statement

A graphic design software package needs to convert an RGB pixel to the CMYK (Cyan, Magenta, Yellow, Black) subtractive color model for printing. The pixel values are $R = 51$, $G = 255$, and $B = 102$.

1. Convert the RGB values to a normalized range $[0,1]$.
2. Calculate the corresponding C, M, Y , and K values using the standard conversion formula:

$$\circ K = 1 - \max(R, G, B)$$

$$\circ C = \frac{1-R-K}{1-K}$$

$$\circ M = \frac{1-G-K}{1-K}$$

$$\circ Y = \frac{1-B-K}{1-K}$$

Solution

នេះជាដំណោះស្រាយលម្អិតមួយជំហានៗសម្រាប់លំហាត់ទី៣ (RGB to CMYK Color Conversion)៖

1. បំប្លែងតម្លៃ RGB ទៅជាចន្លោះ Normalized $[0, 1]$

$$R' = \frac{51}{255} = 0.2, \quad G' = \frac{255}{255} = 1.0, \quad B' = \frac{102}{255} = 0.4$$

2. គណនាតម្លៃ C, M, Y និង K តាមរូបមន្តស្តង់ដារ

- ជំហានទី ៖ គណនាតម្លៃជាតិពណ៌ខ្មៅ Key (K)

$$K = 1 - \max(0.2, 1.0, 0.4) = 1 - 1.0 = 0$$

- ជំហានទី ខ៖ គណនាតម្លៃ Cyan (C)

$$C = \frac{1 - 0.2 - 0}{1 - 0} = \frac{0.8}{1} = 0.8$$

- ជំហានទី គ៖ គណនាតម្លៃ Magenta (M)

$$M = \frac{1 - 1.0 - 0}{1 - 0} = \frac{0}{1} = 0$$

- ជំហានទី ឃ៖ គណនាតម្លៃ Yellow (Y)

$$Y = \frac{1 - 0.4 - 0}{1 - 0} = \frac{0.6}{1} = 0.6$$

សង្ខេបចម្លើយចុងក្រោយ (CMYK):

- Cyan (C) = 0.8 (ត្រូវនឹង 80%)
- Magenta (M) = 0.0 (ត្រូវនឹង 0%)
- Yellow (Y) = 0.6 (ត្រូវនឹង 60%)
- Black (K) = 0.0 (ត្រូវនឹង 0%)

Exercise 4 : HSI intensity Extraction and Enhancement

□ Problem Statement

Increasing the contrast of a color image by applying histogram equalization to individual R, G, and B channels independently creates severe color distortion. The correct approach is to transform the image to a space like HSI (or HSV), equalize only the Intensity/Value channel, and convert it back.

Write a Python script to:

1. Load a color image and convert to the HSV color space.
2. Split the image into individual H, S, and V channels.
3. Apply standard histogram equalization to the V (Value/Intensity) channel using `cv2.equalizeHist()`.
4. Merge the channels back together and convert the image back to the BGR space for saving.

✓ Solution

នេះគឺជាកូដ Python ពេញលេញដែលប្រើប្រាស់បណ្តាញ **OpenCV** និង **NumPy**

ដើម្បីដោះស្រាយលំហាត់ទី៤ ស្របតាមលក្ខខណ្ឌ និងលំដាប់លំដោយទាំង ៤ ជំហានរបស់អ្នក។

បច្ចេកទេសកែច្នៃតន្ត្រីនេះគឺមានសារៈសំខាន់ខ្លាំងណាស់ (ដូចដែលបានរៀបរាប់ក្នុងប្រធានលំហាត់) ពីព្រោះការធ្វើ Histogram Equalization ទៅលើតែ **V Channel** ជួយបង្កើនកម្រិត Contrast នៃរូបភាពឱ្យបានច្បាស់ស្រឡះល្អ ដោយមិនធ្វើឱ្យប្រែប្រួលជាតិពណ៌ដើម (Color Distortion) របស់រូបភាពឡើយ។

```
import cv2

import numpy as np

#
=====

# កុំបំភ្លេច បើ ប្រើប្រាស់ OpenCV ត្រូវ ដំឡើង ជម្រើស HSV (Load Image and Convert
to HSV)

#
=====
```

```

# OpenCV (OpenCV uses BGR)

# Read the image 'input_image.jpg'
image = cv2.imread('input_image.jpg')

if image is None:

    print("\nError: File not found! (Path)
    \n")

    exit()

# Convert BGR to HSV Color Space
hsv_image = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)

#
=====

# Split the image into Channels (Split into H, S, and V
channels)

#
=====

h_channel, s_channel, v_channel = cv2.split(hsv_image)

#
=====

# Histogram Equalization of V Channel (Equalize V
Channel)

#
=====

# cv2.equalizeHist function to equalize the V Channel
histogram

equalized_v = cv2.equalizeHist(v_channel)

#
=====

```



```

# បញ្ចប់ការប្រើប្រាស់ Channels បញ្ចប់ការប្រើប្រាស់ BGR (Merge and
Convert back)

#
=====

# បញ្ចប់ការប្រើប្រាស់ V channel បញ្ចប់ការប្រើប្រាស់ Contrast បញ្ចប់ការប្រើប្រាស់ H បញ្ចប់ការប្រើប្រាស់ S
បញ្ចប់ការប្រើប្រាស់

hsv_enhanced = cv2.merge([h_channel, s_channel, equalized_v])

# បញ្ចប់ការប្រើប្រាស់ HSV បញ្ចប់ការប្រើប្រាស់ BGR បញ្ចប់ការប្រើប្រាស់
បញ្ចប់ការប្រើប្រាស់

result_image = cv2.cvtColor(hsv_enhanced, cv2.COLOR_HSV2BGR)

# បញ្ចប់ការប្រើប្រាស់ (Save Image)

cv2.imwrite('hsv_equalized_output.jpg', result_image)

#
=====

# បញ្ចប់ការប្រើប្រាស់ (Display Results)

#
=====

cv2.imshow('Original Color Image', image)

cv2.imshow('Enhanced Contrast Image (V-Channel Equalized)',
result_image)

print("\nបញ្ចប់ការប្រើប្រាស់ Contrast បញ្ចប់ការប្រើប្រាស់
'hsv_equalized_output.jpg' បញ្ចប់ការប្រើប្រាស់")

# បញ្ចប់ការប្រើប្រាស់

cv2.waitKey(0)

cv2.destroyAllWindows()

```

□ ពន្យល់បន្ថែម៖

- **cv2.split():** ប្រើដើម្បីបំបែក Matrix រូបភាពដែលជា array 3D ឱ្យទៅជា Matrix 2D ចំនួន ៣ ដាច់ដោយឡែកពីគ្នា (H,S,V)។
- **cv2.equalizeHist():** គណនាហ៊ីស្តូក្រាមនៃរូបភាពសង្កេត (ក្នុងលំហាត់នេះគឺ Vchannel) រួចធ្វើការចែកចាយកម្រិតពន្លឺ (Brightness values) ឱ្យរាយប៉ាយស្មើគ្នាពេញចន្លោះ 0 ដល់ 255 ដែលធ្វើឱ្យផ្នែកងងឹតប្រែជាភ្លឺច្បាស់និងផ្នែកដែលស្រអាប់ប្រែជាមាន Contrast ខ្ពស់។
- **cv2.merge():** យក Matrix 2D ទាំង ៣ មកដេរភ្ជាប់គ្នាទៅជាទម្រង់រូបភាព 3-channel វិញ។

Exercise 5 : Dynamic Range Density Slicing

□ Problem Statement

An airport baggage scanner outputs 8-bit grayscale images $f(x, y)$. To highlight hidden illegal goods, write a Python script that applies pseudo-color density slicing:

- Low-density areas ($0 \leq f \leq 60$) map to Pure Blue (BGR: [255, 0, 0])
- Mid-density organic compounds ($61 \leq f \leq 170$) map to Pure Green (BGR: [0, 255, 0])
- High-density metallic shielding ($171 \leq f \leq 255$) map to Pure Red (BGR: [0, 0, 255]).

✓ Solution

នេះគឺជាកូដ Python ពេញលេញដោយប្រើប្រាស់បណ្តាញ **OpenCV** និង **NumPy** ដើម្បីបង្កើតកម្មវិធីកាត់បំបែកកម្រិតដង់ស៊ីតេរូបភាព និងជះពណ៌ក្លែងក្លាយ (**Pseudo-color Density Slicing**) ទៅលើកាបូបធ្វើដំណើរ ស្របតាមលក្ខខណ្ឌដង់ស៊ីតេទាំង ៣ ក្នុងប្រធានលំហាត់របស់អ្នក៖

```
import cv2

import numpy as np

#
=====

# កាត់បំបែក កាបូបធ្វើដំណើរ (Load Grayscale Image)

#
=====

# កាត់បំបែក កាបូបធ្វើដំណើរ Grayscale កាត់បំបែក កាបូបធ្វើដំណើរ
cv2.IMREAD_GRAYSCALE

# កាត់បំបែក កាបូបធ្វើដំណើរ 'baggage_scan.jpg'
កាត់បំបែក កាបូបធ្វើដំណើរ កាត់បំបែក កាបូបធ្វើដំណើរ

gray_img = cv2.imread('baggage_scan.jpg', cv2.IMREAD_GRAYSCALE)

if gray_img is None:

print("\nError: កាត់បំបែក កាបូបធ្វើដំណើរ!
កាត់បំបែក កាបូបធ្វើដំណើរ កាត់បំបែក កាបូបធ្វើដំណើរ\n")
```

```

exit()

# =====

height, width = gray_img.shape

# =====

# Create empty BGR image

# =====

# Matrix 3-channel (BGR)
# (dtype=np.uint8)

color_sliced = np.zeros((height, width, 3), dtype=np.uint8)

# =====

# Apply Density Slicing

# =====

# NumPy Masking (no for loop)

# f. (0 <= f <= 60) -> Pure Blue (BGR: [255, 0, 0])

blue_mask = (gray_img >= 0) & (gray_img <= 60)

color_sliced[blue_mask] = [255, 0, 0]

# g. (61 <= f <= 170) -> Pure Green (BGR: [0, 255, 0])

green_mask = (gray_img >= 61) & (gray_img <= 170)

color_sliced[green_mask] = [0, 255, 0]

```

```

# ១. កាត់ក្រឡាចក្រវិជ្ជា (171 <= f <= 255) -> ក្រឡាចក្រវិជ្ជា Pure Red (BGR:
[0, 0,
255])

red_mask = (gray_img >= 171) & (gray_img <= 255)

color_sliced[red_mask] = [0, 0, 255]

#
=====

# កាត់ក្រឡាចក្រវិជ្ជា ក្រឡាចក្រវិជ្ជា ក្រឡាចក្រវិជ្ជា (Save and Display Results)

#
=====

# កាត់ក្រឡាចក្រវិជ្ជា ក្រឡាចក្រវិជ្ជា ក្រឡាចក្រវិជ្ជា

cv2.imwrite('baggage_sliced_output.jpg', color_sliced)

# កាត់ក្រឡាចក្រវិជ្ជា ក្រឡាចក្រវិជ្ជា ក្រឡាចក្រវិជ្ជា

cv2.imshow('Original Scanner Image (Grayscale)', gray_img)

cv2.imshow('Density Sliced Image (Pseudo-color)', color_sliced)

print("\nកាត់ក្រឡាចក្រវិជ្ជា! កាត់ក្រឡាចក្រវិជ្ជា ក្រឡាចក្រវិជ្ជា ក្រឡាចក្រវិជ្ជា
'baggage_sliced_output.jpg' កាត់ក្រឡាចក្រវិជ្ជា\n")

cv2.waitKey(0)

cv2.destroyAllWindows()

```

១ ការពន្យល់ពីបច្ចេកទេសក្នុងកូដ៖

- **np.zeros((height, width, 3), ...)**: បង្កើតផ្ទាំងរូបភាពទទេមួយជាទម្រង់ពណ៌
- BGR (មាន 3 channels) ដើម្បីត្រៀមឈាបពណ៌ខៀវ បៃតង និងក្រហមទៅលើវា។
- **NumPy Masking (ដូចជា blue_mask)**: ជំនួសឱ្យការប្រើ Loop for
- ពីរជាន់ដើម្បីពិនិត្យម្តងមួយ Pixel (ដែលធ្វើឱ្យកូដងើយឺតខ្លាំង)
- យើងប្រើប្រមាណវិធីគក្កវិជ្ជារបស់ NumPy ដើម្បីបង្កើតផ្ទាំង Matrix នៃតម្លៃ True/False
- (ហៅថា Mask)។ នៅពេលយើងសរសេរ color_sliced[blue_mask] = [255, 0, 0]

- នោះគ្រប់ទីតាំង Pixel ណាដែលមានតម្លៃ True ក្នុងរូបភាពសង្កេតឃើញ
- នឹងត្រូវប្តូរទៅជាពណ៌ខៀវភ្លាមៗក្នុងពេលតែមួយ។
- **ប្រព័ន្ធពណ៌ OpenCV:** ត្រូវចាំថា OpenCV ប្រើលំដាប់លំដោយ **BGR** (Blue, Green, Red) មិនមែន RGB ឡើយ ដូចនេះ៖
- ពណ៌ខៀវគឺ [255, 0, 0]
- ពណ៌បៃតងគឺ [0, 255, 0]
- ពណ៌ក្រហមគឺ [0, 0, 255]

Exercise 6 : Given an input image I as show in matrix below

□ Problem Statement

$$I = \begin{bmatrix} 85 & 35 & 112 & 10 \\ 90 & 75 & 185 & 25 \\ 120 & 220 & 235 & 68 \\ 165 & 70 & 89 & 80 \end{bmatrix}$$

1. Compute the output image smoothing, I_{smooth} , if the filter mask is $H = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$.
2. Compute the edge detection I_{edge} , if the filter mask is $H = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$.

✓Solution

សម្គាល់៖ ដោយសារតែមិនមានបញ្ជាក់ពីប្រភេទ *Padding*

តាមគោលការណ៍ស្តង់ដារគឺយើងគណនាត្រង់ចំណុចណាដែល *Kernel* អាចដាក់គ្របពីលើបានពេញលេញ (*Valid Padding*)។ ដូចនេះ រូបភាពលទ្ធផលនឹងបង្កើតឡើងមានទំហំ 2×2 (មាន ៤ ទីតាំងនៅចំណុចណា)។

1. គណនារូបភាពម៉ត់ (I_{smooth}) ដោយប្រើ **Box Filter**

- ទីតាំងទី១ (ចំណុច 75): គ្របលើ Sub-matrix ខាងលើឆ្វេង

$$\text{Sub-matrix} = \begin{bmatrix} 85 & 35 & 112 \\ 90 & 75 & 185 \\ 120 & 220 & 235 \end{bmatrix}$$

$$\text{Value} = \frac{1}{9} \times (85 + 35 + 112 + 90 + 75 + 185 + 120 + 220 + 235) = \frac{1}{9} \times (1157) \approx 128.56$$

- ទីតាំងទី២ (ចំណុច 185): គ្របលើ Sub-matrix ខាងលើស្តាំ

$$\text{Sub-matrix} = \begin{bmatrix} 35 & 112 & 10 \\ 75 & 185 & 25 \\ 220 & 235 & 68 \end{bmatrix}$$

$$\text{Value} = \frac{1}{9} \times (35 + 112 + 10 + 75 + 185 + 25 + 220 + 235 + 68) = \frac{1}{9} \times (965) \approx 107.22$$

- ទីតាំងទី៣ (ចំណុច 220): គ្របលើ Sub-matrix ខាងក្រោមឆ្វេង

$$\text{Sub-matrix} = \begin{bmatrix} 90 & 75 & 185 \\ 120 & 220 & 235 \\ 165 & 70 & 89 \end{bmatrix}$$

$$\text{Value} = \frac{1}{9} \times (90 + 75 + 185 + 120 + 220 + 235 + 165 + 70 + 89) = \frac{1}{9} \times (1249) \approx 138.78$$

- ទីតាំងទី៤ (ចំណុច 235): គ្របលើ Sub-matrix ខាងក្រោមស្តាំ

$$\text{Sub-matrix} = \begin{bmatrix} 75 & 185 & 25 \\ 220 & 235 & 68 \\ 70 & 89 & 80 \end{bmatrix}$$

$$\text{Value} = \frac{1}{9} \times (75 + 185 + 25 + 220 + 235 + 68 + 70 + 89 + 80) = \frac{1}{9} \times (1047) \approx 116.33$$

ដូចនេះ ផ្ទាំងរូបភាពម៉ត់ចុងក្រោយគឺ៖

$$I_{\text{smooth}} = \begin{bmatrix} 128.56 & 107.22 \\ 138.78 & 116.33 \end{bmatrix} \approx \begin{bmatrix} 129 & 107 \\ 139 & 116 \end{bmatrix}$$

2. គណនារូបភាពចាប់គែម (I_{edge}) ដោយប្រើ Laplacian Filter

រូបមន្ត Laplacian (ដេរីវេទី២)៖ $\text{Value} = (\text{លើ} + \text{ក្រោម} + \text{ឆ្វេង} + \text{ស្តាំ}) - 4 \times (\text{កណ្តាល})$

- **ទីតាំងទី១ (ចំណុច 75):**

$$\text{Value} = (35 + 220 + 90 + 185) - 4(75) = 530 - 300 = 230$$

- **ទីតាំងទី២ (ចំណុច 185):**

$$\text{Value} = (112 + 235 + 75 + 25) - 4(185) = 447 - 740 = -293 \rightarrow 0$$

(តម្លៃ Pixel មិនអាចអវិជ្ជមានបានទេ ដូចនេះត្រូវកាត់ស្មើ 0 តាមគោលការណ៍ Clipping [0,255])

- **ទីតាំងទី៣ (ចំណុច 220):**

$$\text{Value} = (75 + 70 + 120 + 235) - 4(220) = 500 - 880 = -380 \rightarrow 0$$

- **ទីតាំងទី៤ (ចំណុច 235):**

$$\text{Value} = (185 + 89 + 220 + 68) - 4(235) = 562 - 940 = -378 \rightarrow 0$$

ដូចនេះ ផ្ទាំងរូបភាពចាប់គែមចុងក្រោយគឺ៖

$$I_{\text{edge}} = \begin{bmatrix} 230 & 0 \\ 0 & 0 \end{bmatrix}$$

Exercise 7 : The binary image A as shown in figure below

□ Problem Statement

$$A = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 \end{bmatrix} \quad \text{and} \quad B = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

1. Compute $A \oplus B$ (Dilation) and $A \ominus B$ (Erosion).
2. Compute the internal and external boundary.
3. Compute the morphological gradient.

✓ Solution

1. គណនា Dilation $A \oplus B$ និង Erosion $A \ominus B$

- Dilation ($A \oplus B$): គឺពង្រីកតំបន់លេខ 1។ ឱ្យតែផ្ចិតរបស់ B ជាន់ចំលេខ 1 ណាមួយរបស់ A នោះគ្រប់ទីតាំងផ្សេងរបស់ B (លើ, ក្រោម, ឆ្វេង, ស្តាំ) ដែលមានតម្លៃ 1 នឹងត្រូវលាបទៅជា 1 ទាំងអស់។
- Erosion ($A \ominus B$): គឺបង្រួមតំបន់លេខ 1។ លទ្ធផលនឹងបាន 1 លុះត្រាតែគ្រប់ទីតាំងរបស់ B ដែលមានតម្លៃ 1 ត្រូវត្រួតស៊ីគ្នា (Fit) លើលេខ 1 របស់ A ទាំងអស់ (បើប៉ះលេខ 0 តែមួយកន្លែង ផ្ចិតនោះនឹងក្លាយជា 0)។

លទ្ធផលគណនា៖

$$A \oplus B = \begin{bmatrix} 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 \end{bmatrix}$$

$$A \ominus B = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

(សម្គាល់សម្រាប់ Erosion: គ្មានទីតាំងណាដែលមានរាងជាសញ្ញាបូក [1] សុទ្ធនៅក្នុង A ឡើយ ដូច្នេះលទ្ធផល Erosion គឺស្មើ 0 ទាំងអស់)

2. គណនា Internal Boundary និង External Boundary

- Internal Boundary (β_{int}): គឺជាខ្សែបន្ទាត់តែម្តងដែលស្ថិតនៅក្នុងរាងកាយវត្ថុដើម។
 - រូបមន្ត៖ $\beta_{\text{int}}(A) = A - (A \ominus B)$
 - គណនា៖ យក A ដកនឹងលទ្ធផល Erosion ($A \ominus B$):

$$\beta_{\text{int}}(A) = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 \end{bmatrix} - \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 \end{bmatrix}$$

- External Boundary (β_{ext}): គឺជាខ្សែបន្ទាត់គែមដែលព័ទ្ធជុំវិញខាងក្រៅវត្ថុដើម។
 - រូបមន្ត៖ $\beta_{\text{ext}}(A) = (A \oplus B) - A$
 - គណនា៖ យកលទ្ធផល Dilation ($A \oplus B$) ដកនឹង A :

$$\beta_{\text{ext}}(A) = \begin{bmatrix} 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 \end{bmatrix} - \begin{bmatrix} 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 \end{bmatrix}$$

3. គណនា Morphological Gradient

Morphological Gradient (G): គណនាដើម្បីទាញយកខ្សែបន្ទាត់គែមរួម (ក្រាស់ជាងមុន) ដោយរាស់វែងពីគម្លាតរាងការរីក (Dilation) និងការរួម (Erosion) នៃរូបភាព។

- រូបមន្ត៖ $G = (A \oplus B) - (A \ominus B)$
- ការគណនា៖

$$G = \begin{bmatrix} 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 \end{bmatrix} - \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 \end{bmatrix}$$

Exercise 8 : You are given a binary image A and a structuring element

□ Problem Statement

Given a binary image A and a structuring element S (with the origin underlined top-left):

$$A = \begin{bmatrix} 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}_{4 \times 5} \quad \text{and} \quad S = \begin{bmatrix} \underline{1} & 1 \\ 0 & 1 \end{bmatrix}_{2 \times 2}$$

1. Compute the Dilation $A \oplus S$ and the Erosion $A \ominus S$.
2. Compute the internal and the external boundary.
3. Compute the morphology gradient.

✓ Solution

លំហាត់នេះប្រើប្រាស់ Structuring Element មិនស៊ីមេទ្រី និងមានផ្ចិត (Origin) នៅជ្រុងលើខាងឆ្វេង (1)៖

1. គណនា Dilation ($A \oplus S$) និង Erosion ($A \ominus S$)

$$A \oplus S = \begin{bmatrix} 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \end{bmatrix} \quad \text{និង} \quad A \ominus S = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

2. គណនា Internal Boundary និង External Boundary

- **Internal Boundary (β_{int}):**

$$\beta_{\text{int}}(A) = A - (A \ominus S) = \begin{bmatrix} 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix} - \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

- **External Boundary (β_{ext}):**

$$\beta_{\text{ext}}(A) = (A \oplus S) - A = \begin{bmatrix} 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \end{bmatrix} - \begin{bmatrix} 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 \end{bmatrix}$$

3. គណនា Morphological Gradient

$$G = (A \oplus S) - (A \ominus S) = \begin{bmatrix} 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \end{bmatrix} - \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \end{bmatrix}$$